

E²GraphRAG: Streamlining Graph-based RAG for High Efficiency and Effectiveness

Yibo Zhao¹, Jiapeng Zhu¹, Ye Guo², Kangkang He², Xiang Li^{1*}

¹Department of Data Science and Engineering, East China Normal University

²China Baowu Group

{yibozhao, jiapengzhu}@stu.ecnu.edu.cn, xiangli@dase.ecnu.edu.cn
179178@baosteel.com, hekangkang@baowugroup.com

Abstract

Graph-based RAG methods like GraphRAG have shown promising global understanding of the knowledge base by constructing hierarchical entity graphs. However, they often suffer from inefficiency and rely on manually pre-defined query modes, limiting practical use. In this paper, we propose E²GraphRAG, a streamlined graph-based RAG framework that improves both Efficiency and Effectiveness. During the indexing stage, E²GraphRAG constructs a summary tree with large language models and an entity graph with SpaCy based on document chunks. We then construct bidirectional indexes between entities and chunks to capture their many-to-many relationships, enabling fast lookup during both local and global retrieval. For the retrieval stage, we design an adaptive retrieval strategy that leverages the graph structure to retrieve and select between local and global modes. Experiments show that E²GraphRAG achieves up to 10× faster indexing than GraphRAG and 100× speedup over LightRAG in retrieval while maintaining competitive QA performance.

1 Introduction

With the continuous advancement, large language models (LLMs) [5, 28, 42] have become a cornerstone in NLP, which have been widely applied in tasks such as text summarization [16, 25], machine translation [17, 24], and question answering [4, 21, 35]. However, they still face limitations, including hallucinations [7, 31, 32, 37] and a lack of domain-specific knowledge [15, 23, 36, 44]. To address these issues, Retrieval-Augmented Generation (RAG) has been proposed [9, 18, 19]. By retrieving relevant knowledge from external sources and leveraging the in-context learning capabilities of LLMs, RAG allows models to integrate timely and domain-specific information, thereby mitigating issues such as hallucinations and knowledge gaps.

Traditional RAG methods typically retrieve only a small set of chunks from original documents as supplemental knowledge. However, this limited context could be insufficient for providing the model with a comprehensive and global understanding of the knowledge base, such as understanding and summarizing a character’s personality transformation, as in NovelQA [43]. Consider the novel *Harry Potter and the Prisoner of Azkaban* and the question: “*Peter Pettigrew is used to be positive and finally becomes a negative one. Tell in one sentence what marks this character’s change.*” Traditional RAG methods typically retrieve only a few isolated chunks about Peter Pettigrew, whereas answering this question requires a comprehensive understanding of his entire character arc.

To address the problem, existing state-of-the-art methods, including RAPTOR [34], GraphRAG [8], and LightRAG [11], adopt an *indexing-and-retrieval paradigm*: they first use LLMs to index the

*Corresponding Author

documents into tree- or graph-based structures² and then retrieve on these structured data. In particular, during the indexing stage, constructing a hierarchical tree by recursively merging text chunks offers the advantage of global understanding. However, it fails to extract and represent the fine-grained knowledge, i.e., entities and their relations within the text. Further, knowledge graph enables the extraction and integration of fine-grained knowledge in disperse chunks, but it heavily relies on LLMs to extract entities and relations, which incurs high time and computational costs during indexing.

To sum up, existing models suffer from three major issues. First, efficiency is the primary bottleneck of graph-based RAG. While some efforts [2, 11] have been devoted to reducing the overhead, it still remains less than satisfactory. Second, most methods employ either tree or graph structure to organize raw lengthy texts. Despite their own strengths, their integration has yet to be thoroughly explored. Third, in the retrieval stage, some methods [8, 11] require to manually pre-set query modes (e.g., local or global), hence lack flexibility. Therefore, a research question naturally arises: *Can we develop a graph-based RAG model with high efficiency and effectiveness that can adapt to queries at different levels of granularity?*

In this paper, we streamline graph-based RAG for high efficiency and effectiveness, and propose the E²GraphRAG model, which combines the strengths of both tree and graph structures. Specifically, we first recursively merge and summarize text chunks to construct a hierarchical tree structure, which can provide multi-granularity summarization of raw texts. To further integrate fine-grained knowledge from scattered chunks, we also construct a concise entity graph. Instead of using LLMs for entity extraction, we employ the traditional NLP tool SpaCy [14] to extract entities, and use their *co-occurrence* in a chunk as relations. Meanwhile, we build entity-to-chunk and chunk-to-entity indexes to bridge the entity graph and the summary tree, thereby facilitating the lookup process during subsequent retrieval. In the retrieval stage, we introduce a lightweight and adaptive strategy that leverages the entity graph to dynamically select between local and global query modes. Specifically, if the query entities are densely connected within the graph, we perform local retrieval; otherwise, we fall back to global retrieval. This adaptive mechanism enables more efficient and targeted retrieval by explicitly modeling the structural relationships among entities, thereby avoiding setting the tedious query mode and being more flexible to various types of queries.

Finally, we conduct extensive experiments on benchmark datasets to verify the effectiveness and efficiency of our model. Our results show that E²GraphRAG achieves up to 10 $\times$ speedup over GraphRAG in the indexing stage and 100 $\times$ speedup over LightRAG during the retrieval stage, while maintaining state-of-the-art or highly competitive effectiveness.

2 Related Work

RAG has been extensively studied, where most existing methods fall into two main categories based on the type of external knowledge source. Most approaches rely on unstructured textual knowledge bases, which are easy to organize and adaptable to various tasks, but often lack a global and structured understanding of the content. Others utilize structured entity graphs [12, 13, 20, 38], which naturally support multi-hop reasoning and information aggregation for deeper semantic retrieval. However, building high-quality, domain-specific knowledge graphs typically requires substantial expert efforts and is difficult to scale.

GraphRAG [8] is the first method proposed to automatically construct knowledge graphs from raw text and support global query, which attracts considerable attention [27, 48]. It leverages the capabilities of LLMs to construct a knowledge graph from the document, then clusters nodes and summarizes each cluster into higher-level communities, which forms a multi-grained knowledge graph. However, this approach requires numerous LLM calls, leading to the highly expensive indexing step. Moreover, for global retrieval, it relies on the LLM to determine which communities are relevant to the query, resulting in significant latency and computational overhead.

Instead of extracting an entity graph, RAPTOR [34] proposes to construct a hierarchical summary tree by recursively clustering and summarizing the chunks, which avoids the complicated process of entity and relation extraction with LLMs. However, the method ignores the original document’s contextual flow, and the clustering process is also time-consuming. Further, RAPTOR adopts the traditional RAG-style vector-based retrieval, which may lead to inaccurate retrieval results [40].

²Since tree is a special form of graph, we uniformly use graph-based RAG in this paper.

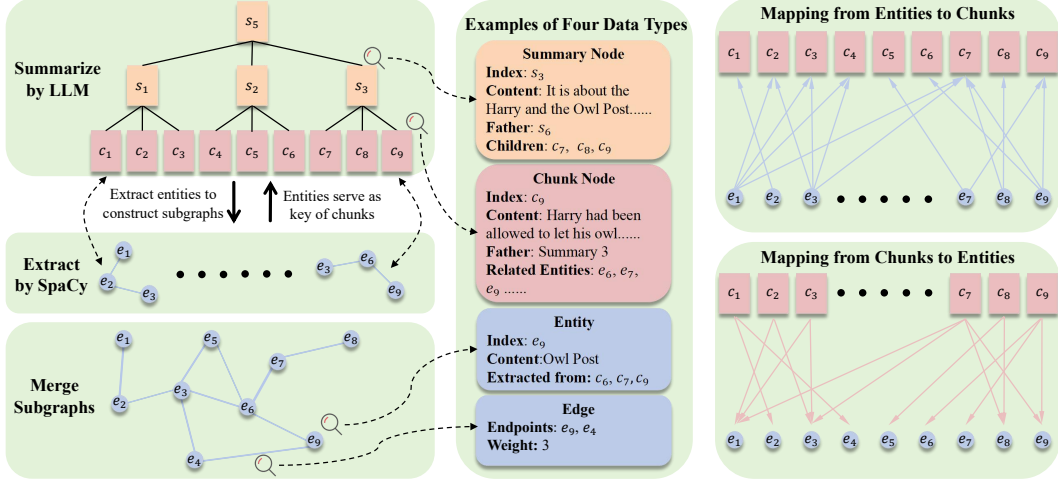


Figure 1: Overview of the indexing stage of E^2 GraphRAG . The left part shows the indexing tasks, the center presents the four data structures, and the right part displays the two constructed indexes.

To improve the efficiency of graph-based RAG, recent methods such as LightRAG [11] and FastGraphRAG [2] aim to reduce the high indexing cost in GraphRAG by removing the clustering and community summarization processes. LightRAG directly prompts the LLM to extract multi-granularity entities and relations from each chunk in a single pass, enabling direct matching during retrieval. FastGraphRAG adopts a similar extraction strategy during indexing but instead applies a variant of PageRank [26] at retrieval time to support global retrieval without relying on community structures. While both approaches reduce indexing overhead to some extent, they still require the LLM to generate complex and verbose JSON-format outputs for each chunk, resulting in considerable time and resource costs. Further, LazyGraphRAG [1] defers all LLM calls to the retrieval stage. Although it reduces the indexing burden, it incurs high latency during retrieval, as it requires multiple LLM invocations to construct subgraphs for a single query.

3 Method

Similar to GraphRAG and other methods, our approach consists of two main stages: **indexing** and **retrieval**. For our task, we first introduce some symbolic definitions to facilitate clearer explanations in the subsequent section. As input, we use D to represent the document, q denotes the query, and k denotes the max chunks retrieved.

3.1 Indexing Stage

As in standard RAG indexing, we first split each document into n chunks. We tokenize the document using the tokenizer corresponding to the model used in the subsequent summarization task, and divide it into chunks of 1200 tokens each, with an overlap of 100 tokens between adjacent chunks to mitigate the semantic loss caused by potential sentence fragmentation. The resulting chunked document is denoted as $D = \{c_1, c_2, \dots, c_n\}$. Then, as illustrated in Figure 1, the indexing stage comprises two main tasks: construction of a **summary tree** and extraction of an **entity graph**. To enhance subsequent retrieval, we further introduce two types of indexes that establish many-to-many mappings between the tree and the graph.

For the summary tree construction, we preserve the original chunk order and employ an LLM to summarize every consecutive group of g chunks. Notably, since most modern LLMs have been extensively trained on text summarization tasks during the instruction tuning [33, 45], we adopt a minimal prompting strategy—providing only task instructions without lengthy few-shot examples, as required in LightRAG and GraphRAG—thereby improving indexing efficiency. Once all the original chunks have been summarized, the resulting summaries are treated as a new sequence of inputs. This recursive summarization process continues, grouping every g summaries at each level, until only g or fewer segments remain. Through the above procedure, the raw document is transformed into a tree

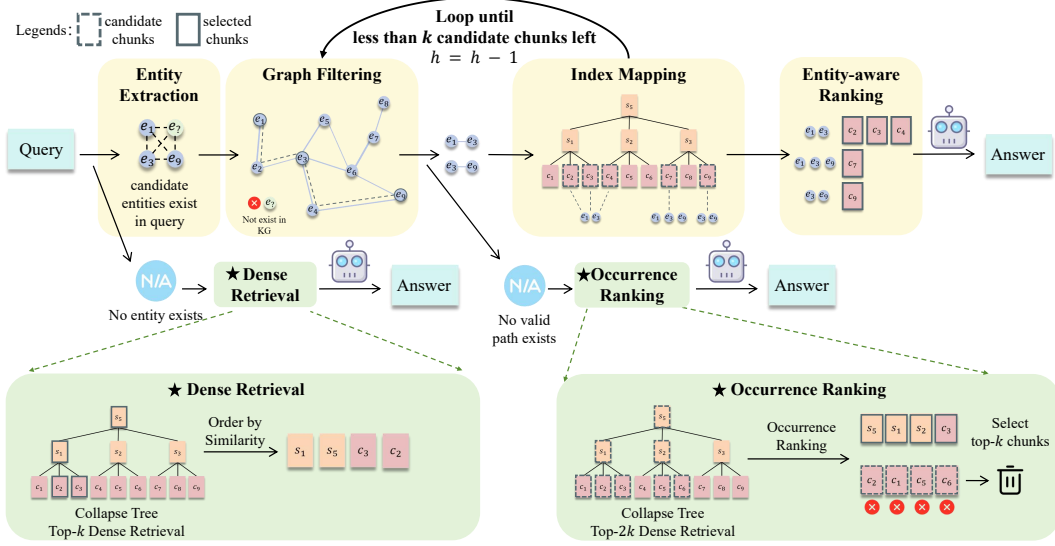


Figure 2: The retrieval stage of E²GraphRAG. Operations belonging to the local retrieval are highlighted in light yellow, while those for global retrieval are highlighted in light green.

structure, where the leaf nodes correspond to chunks and the intermediate or root nodes correspond to the summaries. Nodes closer to the root contain more global and abstract information, while those nearer to the leaves retain more detailed and specific content. We then utilize a pretrained embedding model to encode all chunks and summaries, storing the resulting vectors using *Faiss* [6] to enable efficient dense retrieval. Formally, we denote the summary tree as $T = \{c_1, \dots, c_n, s_1, \dots, s_o\}$, where each chunk c_i and summary s_i corresponds to a node in the tree.

For the entity graph extraction task, instead of relying on LLMs to extract entities and relations as in GraphRAG-style approaches, we opt for a more efficient strategy by leveraging the traditional NLP toolkit SpaCy [14], which is well-suited for large-scale information extraction. In particular, we extract named entities and common nouns (as nouns often indicate potential entities), and uniformly refer to them as *entities* hereafter. Formally, for each chunk c_i , we denoted the extracted entities as $\mathcal{E}_{c_i} = \{e_1^i, \dots, e_m^i\}$, where m is the number of entities identified in chunk c_i . After extracting entities, we construct an undirected weighted edge between any two entities that co-occur within the same sentence, where the edge weight reflects their sentence-level co-occurrence frequency. This results in a subgraph $\mathcal{G}_{c_i}$ for each chunk c_i , which captures the relations among entities mentioned within the chunk and allows us to construct associations between entities and chunks. To support efficient retrieval, we build two one-to-many indexes to link entities and chunks and reflect the many-to-many relations between them. The entity-to-chunk index, $I_{e \rightarrow c}(\cdot)$, maps each entity to the set of chunks where it appears. The chunk-to-entity index, $I_{c \rightarrow e}(\cdot)$, records the entities extracted from each chunk. These two indexes establish a many-to-many mapping between the entities in the entity graph and the chunks in the summary tree, facilitating the subsequent entity-aware retrieval stage. For the entire document, we merge all chunk-level subgraphs into a single graph $\mathcal{G}$, where identical entities are unified and edges with the same source and target entities have their weights summed. Since some entities appear in multiple chunks, this merging allows the graph to capture the co-occurrence relationships among entities across the entire document.

Since the summarization task relies on the LLM and utilizes GPUs, while the entity extraction with SpaCy primarily runs on the CPU, these tasks can be executed in parallel to optimize overall computation time, further reducing the time cost of the indexing stage.

In conclusion, as illustrated in Fig. 1, our method involves four types of data stored in two data structures: summary nodes and original chunk nodes in the tree, along with entities and weighted edges in the graph. In addition, our method relies on two key indexes, chunk-to-entity index $I_{c \rightarrow e}(\cdot)$ and entity-to-chunk index $I_{e \rightarrow c}(\cdot)$, which bridge the tree and the graph. These indexes enable efficient mapping from a chunk to its associated entities, and from an entity to the chunks in which it appears, respectively, thereby facilitating subsequent retrieval.

3.2 Retrieval Stage

In the retrieval stage, previous work faces two main challenges: (1) global queries heavily rely on LLMs, resulting in high retrieval latency, and (2) the retrieval hierarchy and methods often require manual specification, introducing additional hyperparameters that are difficult to optimize. To address these issues, we first introduce a novel retrieval mechanism that adaptively selects between global and local retrieval when specific logical conditions are met. Then, we rank and format the retrieved pieces of evidence therefore enhancing the LLM. To clearly distinguish between the two adaptively selected retrieving modes, we highlight **global retrieval** starting with a ★ throughout this section. The complete pseudocode is provided in Appendix A, and an overview of our retrieval and ranking pipeline is shown in Figure 2.

At the core of our approach is the intuition that each local query typically involves relevant entities, like “*Slytherin*” and “*House Cup*” in the question “*Has Slytherin won the House Cup?*”, and potential relationships among these entities can guide the retrieval process by identifying the most relevant chunks. Therefore, we first use SpaCy, as in the indexing stage, to extract entities from the query, denoted as $\mathcal{E}_q = \{e_q^1, \dots, e_q^m\}$. The entities in the query are then mapped to the vertices in our constructed graph. For simplicity, entities that cannot be mapped to any graph vertex are treated as invalid and ignored, as they are likely noise introduced by erroneous extraction from SpaCy.

★If no entities are identified, we cannot utilize the entities to support meaningful retrieval. In such cases, the query is treated as a global query, and *Dense Retrieval* is performed over the summary tree. Specifically, we adopt a collapsed-tree dense retrieval approach similar to RAPTOR [34], leveraging the embedding model used in the indexing stage to encode the query. The similarity between the query embedding and these indexed embeddings is then computed to select the top- k most relevant chunks as supplementary information, which are ranked in descending order of similarity.

Otherwise, since SpaCy lacks the ability to capture semantic relevance, it often fails to identify the core entities aligned with the query intent, resulting in noisy extractions. Simply mapping these entities to the graph is insufficient for filtering out the noise. Therefore, we introduce a *Graph Filtering* step to retain only the core entities for effective retrieval. The underlying heuristic is that truly relevant entities tend to be semantically related and thus connected in the constructed graph. Formally, they should lie within h hops of each other as neighbors. Specifically, we enumerate all pairwise combinations of entities from the query as candidate entity pairs. For each pair, if the two entities are within h hops in the knowledge graph, they are considered semantically related and retained; otherwise, they are discarded as likely irrelevant. The set of selected entity pairs is denoted as $\mathcal{P}_h$. This step is formally defined in Eq. (1), where $\text{Dist}_G(\cdot, \cdot)$ returns the hop count of the shortest path between two entities in the graph. If no path exists, it returns infinity. The hyperparameter h controls the strictness of the filtering and can be adaptively adjusted to balance the number of chunks recalled during the following steps.

$$\mathcal{P}_h = \{(e_q^i, e_q^j) \in \mathcal{E}_q \times \mathcal{E}_q \mid i < j, \text{Dist}_G(e_q^i, e_q^j) \leq h\} \quad (1)$$

★After this filtering step, if no entity pairs meet the criteria, i.e., there are no fine-grained, interrelated entities in the query, which means their relations cannot be extracted within several local chunks. In such cases, we classify it as a coarse-grained global query as well. This also includes cases where the query contains only a single entity, as there are no pair-wise combination. However, unlike the previous scenario, entities related to both question and context are still present and can assist in improving chunk selection. To leverage them, we first retrieve the top- $2k$ chunks from the summary tree based on vector similarity as candidate supplementary chunks. We then apply an *Occurrence Ranking* strategy, ranking these candidate chunks according to the frequency of entity occurrences, defined as $w(c_i) = \text{Count}(c_i, \mathcal{E}_q)$. For each candidate summary node, the weight is recursively computed as the sum of the weights of its child nodes, i.e. $w(s_i) = \sum_{c/s \in T_{\text{child}}(s_i)} w(c/s)$, where c/s may refer to either chunk nodes or summary nodes. This recursive weighting naturally assigns higher scores to high-level summary nodes, aligning with the intuition behind global retrieval. Finally, we rank the candidate chunks by their computed weights and select the top- k highest-ranked ones as supplementary information.

If entity pairs exist, this indicates the presence of fine-grained relational entities in the query. In such cases, we perform *Index Mapping*, leveraging the entity-to-chunk index $I_{e \rightarrow c}$ constructed during the indexing stage. Specifically, for each entity pair (e_i, e_j) in $\mathcal{P}_h$, we map each entity to the corresponding sets of chunks through the index, and then take their intersection to identify the set of

chunks associated with both entities, denoted as $\mathcal{C}_{\text{evidence}}^{(e_i, e_j)}$. $\mathcal{C}_{\text{evidence}}$, the union of the $\mathcal{C}_{\text{evidence}}^{(e_i, e_j)}$ means all the candidate chunks. Formally, we define the *Index Mapping* operation with Eq. (2).

$$\mathcal{C}_{\text{evidence}} = \bigcup_{(e_i, e_j) \in \mathcal{P}_h} \mathcal{C}_{\text{evidence}}^{(e_i, e_j)} = \bigcup_{(e_i, e_j) \in \mathcal{P}_h} \{I_{e \rightarrow c}(e_i) \cap I_{e \rightarrow c}(e_j)\} \quad (2)$$

Once the indexes are mapped, if the number of retrieved chunks does not exceed k , we directly return them as the final evidence set. Otherwise, we first attempt to reduce the number of chunks by decreasing the hop threshold h step-by-step, as tighter structural constraints help eliminate less relevant neighbors. This continues until either the number of chunks drops below k , or the retrieval returns no chunks at all. If the latter occurs (i.e., the retrieval result becomes empty), we revert to the last non-empty result before the drop and apply an *Entity-Aware Ranking* mechanism to select the top- k chunks from it. This ranking is based on multiple structural and statistical signals derived during retrieval. Specifically, we compute two metrics for each candidate chunk: **Entity Coverage Ranking** counts the number of distinct query-related entities present in the chunk. Chunks covering more entities are prioritized as they are not only more likely to be relevant but also tend to contain more comprehensive contextual information. **Entity Occurrence Ranking** ranks the chunks by the total frequency of query-related entities, which is the same as the *Occurrence Ranking*. Chunks are ranked by these metrics in sequence, first by entity coverage, then by entity occurrence, and the top- k are selected as supplementary evidence. This operation can be facilitated by the chunk-to-entity index $I_{c \rightarrow e}(\cdot)$ to minimize the time cost.

After retrieving all relevant chunks, we proceed to rank and format the chunks and entities as supplementary input to the LLM. Following the earlier intuition that entities serve to highlight the key information while chunks provide the supporting details, we organize the retrieved evidence in an “entity1-entity2: chunks” format. To further reduce token consumption, we apply two optimization strategies. First, to eliminate redundant input caused by chunks associated with multiple entity pairs, we consolidate these chunks into a single format such as “entity1-entity2-...-entity n : chunks”. This de-duplication step ensures that each chunk is included only once, even if it is linked to multiple entity pairs. Second, we detect and merge continuous chunks within the evidence set to eliminate overlaps between adjacent chunks. This chunk merging step further reduces input redundancy and helps minimize token costs. Finally, we rank the entity pairs based on entity coverage and arrange their corresponding chunks according to their original chunk order in the document.

4 Experiment

4.1 Experiment Settings

We describe our experimental setup, including the choice of base models, datasets, and evaluation metrics. For each component, we detail both the selection criteria and the rationale behind, aiming to ensure the reproducibility, practicality, and fairness of our evaluation.

Base Models We choose the Qwen2.5-7B-Instruct[30] and Llama3.1-8B-Instruct [10] as our base model. This decision is motivated by the practical constraints faced by individuals with limited resources and organizations with strict data privacy requirements, for whom accessing large models via costly APIs is often infeasible. In contrast, open-source, relatively lightweight models offer a more economical and widely applicable alternative. Therefore, we conduct our experiments on these models. We adopt the BGE-M3 [3], a state-of-the-art open-source embedding model known for its promising performance, as our embedding backbone.

Datasets We utilize QA datasets constructed from extremely long documents, including NovelQA [43], and a subset of InfiniteBench [47]. Specifically, we select the English multiple-choice and English QA subsets from InfiniteBench, referred to as InfiniteChoice and InfiniteQA, respectively. Each document in these datasets contains, on average, approximately 200k tokens, which enables us to assess the effectiveness of our method and baselines in performing global queries over extremely long documents. More details about these datasets are provided in Appendix B. We do not adopt the UltraDomain [29] used in the LightRAG due to concerns regarding the reliance on LLM-as-judge evaluation [39, 41, 46] and the reliability of traditional metrics when handling relatively long answers. To ensure a more accurate and interpretable evaluation, we follow the task setting of RAPTOR [34], focusing on close-ended QA and multiple-choice tasks.

Table 1: Overall results on NovelQA, InfiniteChoice and InfiniteQA, the best results are highlighted in bold and the runner up result with underline. Met. means the metric for each dataset, we use accuracy for NovelQA and Infinite Choice, Rouge-L for InfiniteQA. IT means indexing time, and QT means querying time.

Backbone Model		Qwen2.5-7B-Instruct			Llama3.1-8B-Instruct		
Dataset		NovelQA	InfiniteChoice	InfiniteQA	NovelQA	InfiniteChoice	InfiniteQA
GraphRAG-L	Met. $\uparrow$	<u>43.34</u>	46.72	<u>13.51</u>	43.64	43.66	<u>6.37</u>
	IT $\downarrow$	13793.89	11816.15	15686.53	4517.09	3921.95	5533.68
	QT $\downarrow$	0.20	0.25	0.82	0.43	0.41	1.16
GraphRAG-G	Met. $\uparrow$	17.48	18.78	2.32	10.93	9.17	1.98
	IT $\downarrow$	13793.89	11816.15	15686.53	4517.09	3921.95	5533.68
	QT $\downarrow$	15.72	16.65	2.83	3.25	3.86	3.33
LightRAG	Met. $\uparrow$	38.57	<u>45.41</u>	10.41	21.82	20.52	3.44
	IT $\downarrow$	5290.93	4732.98	6976.55	3416.31	3225.94	5231.11
	QT $\downarrow$	15.68	16.03	15.97	11.44	12.92	15.44
RAPTOR	Met. $\uparrow$	37.27	34.93	6.42	40.48	37.12	5.83
	IT $\downarrow$	<u>2847.25</u>	<u>2568.26</u>	<u>3407.41</u>	<u>2874.65</u>	<u>2551.89</u>	<u>2844.55</u>
	QT $\downarrow$	0.02	<u>0.08</u>	0.03	0.02	0.03	0.03
E ² GraphRAG	Met. $\uparrow$	45.60	43.23	13.65	<u>41.26</u>	<u>39.74</u>	11.07
	IT $\downarrow$	1397.11	1244.56	1630.87	1641.49	1433.74	1839.26
	QT $\downarrow$	0.02	0.05	0.03	<u>0.03</u>	<u>0.05</u>	0.03

Metrics As discussed above, we focus on the multiple-choice and close-ended QA tasks. Accordingly, we adopt accuracy and ROUGE-L [22] as the evaluation metrics for these two types of QA, respectively. In addition, to assess system efficiency during the indexing and retrieval stage, we measure the indexing time per book and retrieval time per query.

4.2 Baselines

We conduct comparisons against all publicly available and open-source methods to ensure a comprehensive evaluation. The selected baselines include GraphRAG-Local, GraphRAG-Global, LightRAG-Hybrid, and RAPTOR, covering representative approaches. For the summary tree-based RAPTOR, we aligned its prompting format with ours for a fair comparison. In contrast, since LightRAG and GraphRAG require graph extraction with predefined JSON formats and examples, we directly adopted their default prompts. Given the vulnerability of relatively smaller models when extracting entities and producing standard JSON outputs, we set up retries for LightRAG and GraphRAG to ensure their execution. As discussed in the Related Work section, LazyGraphRAG has not released its code, making it infeasible for inclusion in our experiments. For FastGraphRAG, although its code is publicly available, it is incompatible with locally deployed 7B–8B models, since we are unable to instruct the LLM to output the required structured JSON extraction even with five retries for each LLM call. Therefore, we focus on open-source and practically reproducible baselines in our comparisons. More implementation details of the baseline methods can be found in Appendix C.

4.3 Experimental Results

As illustrated in Table 1, E²GraphRAG achieves the highest efficiency in the indexing stage, being **up to $10 \times$ faster than GraphRAG** and approximately **$2 \times$ faster than RAPTOR**, the second fastest method. For the retrieval stage, E²GraphRAG also demonstrates superior speed, achieving **over $100 \times$ speedup compared to LightRAG** and **about $10 \times$ faster than GraphRAG’s local mode**. At the same time, E²GraphRAG maintains competitive effectiveness compared to GraphRAG. In particular, E²GraphRAG achieves the best performance on NovelQA when using Qwen, and on InfiniteQA across both two backbone models.

In terms of efficiency, RAPTOR is the most efficient among the baseline methods. It also achieves the fastest retrieval speed, tied with E²GraphRAG, thanks to dense retrieval accelerated by GPU. However, its effectiveness is unsatisfactory, ranking among the lowest performers, 8% lower than E²GraphRAG on NovelQA with Qwen specifically.

In terms of effectiveness, GraphRAG with local mode outperforms all baseline methods. However, its indexing stage takes approximately 4 hours to process a 200k-token book, making it impractical

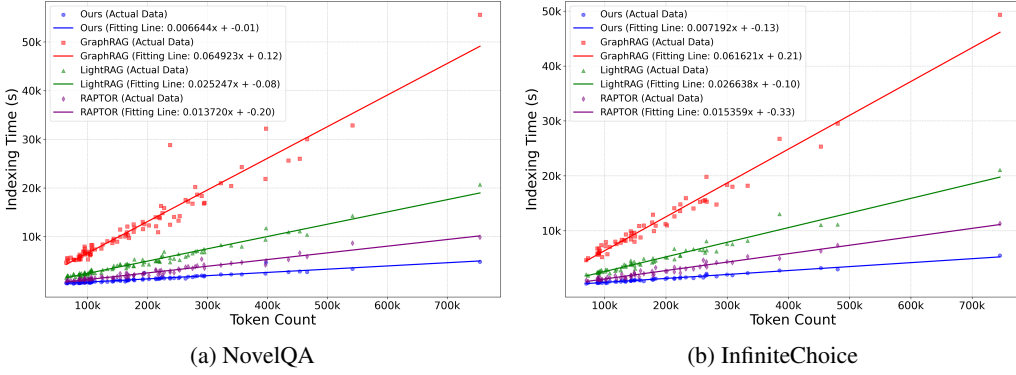


Figure 3: Time cost as a function of document token count for each method. The statistic is based on NovelQA and InfinteChoice with Qwen as the base model.

for real-world use. This inefficiency is primarily caused by the instability of JSON output from small LLMs, particularly with Qwen, which is significantly slower than Llama. Apart from the same inefficient indexing operation, GraphRAG in global mode shows inferior performance due to two primary reasons: (1) the aggregation of global context introduces semantic noise, potentially incorporating irrelevant information; and (2) the LLM fails to provide the required JSON format to choose communities to support answering.

LightRAG tries to strike a balance between effectiveness and efficiency, but remains inadequate. Extracting multi-grained entities and relations from each chunk leads to high latency and depends heavily on the LLM’s capabilities, failing especially when using Llama3.1.

These results demonstrate that E²GraphRAG achieves the best trade-off between efficiency and effectiveness among all compared methods, making it a practical solution for real-world applications.

4.4 Computational Cost Analysis

In addition to the wall-time comparison reported in Table 1, we provide a more intuitive visualization of indexing efficiency in Figure 3, which presents scatter plots of indexing time across varying document lengths based on the Qwen model. Each method is fitted with a linear function to highlight the differences in time overhead, indicating that our method scales linearly with the lowest slope among all methods. Furthermore, to better understand the computational burden behind these results, we estimate the theoretical costs to assess the computational overhead. Since the primary cost during the indexing stage stems from LLM inference, we calculated the number of LLM calls required by each method during this phase. For the retrieval stage, each method incurs computational overhead from different sources. Therefore, we qualitatively list the primary sources of overhead without conducting a direct mathematical comparison. Let each document consist of n chunks, and let g denote the number of child nodes aggregated per summarization call. For local and global GraphRAG, we use c to represent the number of extracted communities, and C_{window} denotes the length of the context window of the corresponding LLM. Detailed analysis can be found in Appendix D.

E²GraphRAG : During indexing, we construct a single document tree using the LLM, with n leaf nodes. The total number of LLM calls to generate non-leaf nodes is approximately $\lceil n/(g-1) \rceil$. For the retrieval stage, no LLMs are involved, the main computational cost comes from utilizing SpaCy to extract entities from the question.

RAPTOR: RAPTOR similarly constructs a document tree but introduces additional overhead due to clustering, with an unfixed number of chunks per cluster. Its theoretical **lower bound** on LLM calls is $\lceil n/(g-1) \rceil$, slightly higher than our method in theory. In practice, however, clustering produces fragmented structures, resulting in increased latency. For the retrieval stage, RAPTOR performs dense retrieval over the tree, requiring $\lceil n/(g-1) \rceil + n$ vector multiplications.

LightRAG: LightRAG invokes the LLM once per chunk during indexing, resulting in a total of n LLM calls. For the retrieval stage, it calls LLM once for entity recognition, followed by dense retrieval over the constructed knowledge graph.

Table 2: Ablation study results. The best results for each dataset are highlighted in bold. For other methods, the performance difference compared to E²GraphRAG is annotated below each value, with ↓ (in red) indicating a decrease and ↑ (in green) indicating an increase. The annotated numbers represent the absolute difference in performance relative to E²GraphRAG .

Dataset Metric	NovelQA Acc.	InfiniteChoice Acc.	InfiniteQA R-L
E ² GraphRAG	45.38	43.23	13.65
Dense Retrieval Only	42.00 (↓ 3.38)	41.04 (↓ 2.19)	10.03 (↓ 3.62)
w/o Graph Filter	44.30 (↓ 1.08)	36.68 (↓ 6.55)	10.47 (↓ 3.18)
w/o Entity-Aware Ranking	44.12 (↓ 1.26)	40.17 (↓ 3.06)	8.25 (↓ 5.40)
w/o Graph Filter & Entity-Aware Ranking	44.08 (↓ 1.30)	35.81 (↓ 5.23)	10.58 (↓ 3.07)
w/o Dense Retrieval	45.90 (↑ 0.52)	37.99 (↓ 5.24)	13.03 (↓ 0.62)
w/o Occurrence Ranking	44.25 (↓ 1.13)	37.99 (↓ 5.24)	8.39 (↓ 5.16)
w/o Dense Retrieval & Occurrence Ranking	45.33 (↓ 0.05)	37.55 (↓ 5.68)	11.07 (↓ 2.58)

GraphRAG: GraphRAG calls the LLM n times for extracting entities and relations and additional c times for aggregating communities, i.e., $c + n$ LLM calls in total. The retrieval stage in local mode directly uses the question text without entity recognition to retrieve the most relevant entities in the constructed knowledge graph. For the retrieval stage in global mode, it leverages the LLM to decide which communities should be considered, which invokes $c \times \text{len}(c) / C_{\text{window}}$ times of LLM calls.

4.5 Ablation Study

To thoroughly evaluate the contribution of each component in E²GraphRAG , we conduct a comprehensive ablation study on three datasets using the Qwen model. The results are summarized in Table 2, which consists of three main sections:

Baseline Dense Retrieval Only: To verify the necessity and effectiveness of our retrieval strategy, we compare E²GraphRAG with a baseline that relies solely on dense retrieval using the BGE-M3 embedding model and the built summary tree. The results demonstrate that E²GraphRAG significantly outperforms this baseline, validating the importance of our retrieval enhancements.

Local Retrieval Ablations: To assess the impact of the local retrieval components, we individually and jointly ablate the *Graph Filter* and *Entity-Aware Ranking* modules. Results show that both modules are crucial for local evidence selection. The removal of either leads to a significant performance drop, confirming their complementary roles.

Global Retrieval Ablations: Similarly, we evaluate the contribution of the global retrieval by ablating *Dense Retrieval* and *Occurrence Ranking*. Among these, Occurrence Ranking appears more impactful, likely due to its more frequent use in our datasets. Interestingly, we observe an anomalous improvement when removing Dense Retrieval on NovelQA. We hypothesize that this is caused by occasional hallucinations, where the model guesses the correct answer without actual evidence.

5 Conclusion

In this paper, we addressed the inefficiency of existing graph-based RAG methods that hinders their practicality. We streamlined the graph-based RAG pipeline and propose E²GraphRAG . During the indexing stage, we recursively built document summary trees with LLMs and efficiently extracted entity-level knowledge graphs using SpaCy, significantly reducing time costs and improving practicality. In the retrieval stage, we proposed an adaptive strategy that leverages the graph structure to locate relevant chunks and automatically select between local and global retrieval modes, eliminating the need for manually pre-defined query settings. By combining the summary tree and knowledge graph, E²GraphRAG enables adaptive global and local retrieval. Extensive experiments demonstrate that E²GraphRAG achieves state-of-the-art efficiency in both indexing and retrieval stages, with up to $10\times$ speedup over GraphRAG in indexing and $100\times$ over LightRAG in retrieval, while maintaining competitive effectiveness.

References

- [1] Lazygraphrag: Setting a new standard for quality and cost. <http://https://www.microsoft.com/en-us/research/blog/lazygraphrag-setting-a-new-standard-for-quality-and-cost/>.
- [2] Fastgraphrag. <https://github.com/circlemind-ai/fast-graphrag>, 2024.
- [3] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In L.-W. Ku, A. Martins, and V. Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2318–2335, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics.
- [4] X. Chen, X. Chen, B. He, T. Wen, and L. Sun. Analyze, generate and refine: Query expansion with LLMs for zero-shot open-domain QA. In L.-W. Ku, A. Martins, and V. Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11908–11922, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics.
- [5] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: fast and memory-efficient exact attention with io-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2022. Curran Associates Inc.
- [6] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou. The faiss library. 2024.
- [7] X. Du, C. Xiao, and Y. Li. Haloscope: Harnessing unlabeled llm generations for hallucination detection. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 102948–102972. Curran Associates, Inc., 2024.
- [8] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitan, R. O. Ness, and J. Larson. From local to global: A graph rag approach to query-focused summarization, 2025.
- [9] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD ’24, page 6491–6501, New York, NY, USA, 2024. Association for Computing Machinery.
- [10] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov,

- T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, et al. The llama 3 herd of models, 2024.
- [11] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang. Lightrag: Simple and fast retrieval-augmented generation, 2024.
 - [12] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su. Hipporag: Neurobiologically inspired long-term memory for large language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 59532–59569. Curran Associates, Inc., 2024.
 - [13] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
 - [14] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd. spaCy: Industrial-strength Natural Language Processing in Python. 2020.
 - [15] Z. Jiang, L. Zhong, M. Sun, J. Xu, R. Sun, H. Cai, S. Luo, and Z. Zhang. Efficient knowledge infusion via KG-LLM alignment. In L.-W. Ku, A. Martins, and V. Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2986–2999, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics.
 - [16] F. Kirstein, T. Ruas, R. Kratel, and B. Gipp. Tell me what I need to know: Exploring LLM-based (personalized) abstractive multi-source meeting summarization. In F. Dernoncourt, D. Preotiuc-Pietro, and A. Shimorina, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 920–939, Miami, Florida, US, Nov. 2024. Association for Computational Linguistics.
 - [17] R. Koshkin, K. Sudoh, and S. Nakamura. TransLLaMa: LLM-based simultaneous translation system. In Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 461–476, Miami, Florida, USA, Nov. 2024. Association for Computational Linguistics.
 - [18] P. Laban, A. Fabbri, C. Xiong, and C.-S. Wu. Summary of a haystack: A challenge to long-context LLMs and RAG systems. In Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9885–9903, Miami, Florida, USA, Nov. 2024. Association for Computational Linguistics.
 - [19] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.
 - [20] D. Li, S. Yang, Z. Tan, J. Y. Baik, S. Yun, J. Lee, A. Chacko, B. Hou, D. Duong-Tran, Y. Ding, H. Liu, L. Shen, and T. Chen. DALK: Dynamic co-augmentation of LLMs and KG to answer Alzheimer’s disease questions with scientific literature. In Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 2187–2205, Miami, Florida, USA, Nov. 2024. Association for Computational Linguistics.
 - [21] Z. Li, S. Fan, Y. Gu, X. Li, Z. Duan, B. Dong, N. Liu, and J. Wang. Flexkbqa: A flexible llm-powered framework for few-shot knowledge base question answering. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):18608–18616, Mar. 2024.
 - [22] C.-Y. Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics.
 - [23] J. Liu, C. Zhang, J. Guo, Y. Zhang, H. Que, K. Deng, Z. Bai, J. Liu, G. Zhang, J. Wang, Y. Wu, C. Liu, J. Wang, L. Qu, W. Su, and B. Zheng. Ddk: Distilling domain knowledge for efficient large language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 98297–98319. Curran Associates, Inc., 2024.

- [24] Y. Lu, W. Zhu, L. Li, Y. Qiao, and F. Yuan. LLaMAX: Scaling linguistic horizons of LLM by enhancing translation capabilities beyond 100 languages. In Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10748–10772, Miami, Florida, USA, Nov. 2024. Association for Computational Linguistics.
- [25] N. Nakshatri, S. Liu, S. Chen, D. Roth, D. Goldwasser, and D. Hopkins. Using LLM for improving key event discovery: Temporal-guided news stream clustering with event summaries. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 4162–4173, Singapore, Dec. 2023. Association for Computational Linguistics.
- [26] L. Page, S. Brin, R. Motwani, and T. Winograd. The pagerank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab, November 1999. Previous number = SIDL-WP-1999-0120.
- [27] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, and S. Tang. Graph retrieval-augmented generation: A survey, 2024.
- [28] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference. In D. Song, M. Carbin, and T. Chen, editors, *Proceedings of Machine Learning and Systems*, volume 5, pages 606–624. Curan, 2023.
- [29] H. Qian, Z. Liu, P. Zhang, K. Mao, D. Lian, Z. Dou, and T. Huang. Memorag: Boosting long context processing with global memory-enhanced retrieval augmentation. In *Proceedings of the ACM Web Conference 2025 (TheWebConf 2025)*, Sydney, Australia, 2025. ACM. arXiv:2409.05591.
- [30] Qwen, :, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report, 2025.
- [31] S. Ramprasad, E. Ferracane, and Z. Lipton. Analyzing LLM behavior in dialogue summarization: Unveiling circumstantial hallucination trends. In L.-W. Ku, A. Martins, and V. Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12549–12561, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics.
- [32] P. Sahoo, P. Meharia, A. Ghosh, S. Saha, V. Jain, and A. Chadha. A comprehensive survey of hallucination in large language, image, video and audio foundation models. In Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 11709–11724, Miami, Florida, USA, Nov. 2024. Association for Computational Linguistics.
- [33] V. Sanh, A. Webson, C. Raffel, S. Bach, L. Sutawika, Z. Alyafeai, A. Chaffin, A. Stiegler, A. Raja, M. Dey, M. S. Bari, C. Xu, U. Thakker, S. S. Sharma, E. Szczechla, T. Kim, G. Chhablani, N. Nayak, D. Datta, J. Chang, M. T.-J. Jiang, H. Wang, M. Manica, S. Shen, Z. X. Yong, H. Pandey, R. Bawden, T. Wang, T. Neeraj, J. Rozen, A. Sharma, A. Santilli, T. Fevry, J. A. Fries, R. Teehan, T. L. Scao, S. Biderman, L. Gao, T. Wolf, and A. M. Rush. Multi-task prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*, 2022.
- [34] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning. Raptor: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024.
- [35] T. Schimanski, J. Ni, M. Kraus, E. Ash, and M. Leippold. Towards faithful and robust LLM specialists for evidence-based question-answering. In L.-W. Ku, A. Martins, and V. Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1913–1931, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics.
- [36] J. Shen, N. Tenenholtz, J. B. Hall, D. Alvarez-Melis, and N. Fusi. Tag-llm: repurposing general-purpose llms for specialized domains. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org, 2024.

- [37] G. Sriramanan, S. Bharti, V. S. Sadasivan, S. Saha, P. Kattakinda, and S. Feizi. Llm-check: Investigating detection of hallucinations in large language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 34188–34216. Curran Associates, Inc., 2024.
- [38] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. Ni, H.-Y. Shum, and J. Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *The Twelfth International Conference on Learning Representations*, 2024.
- [39] A. Szymanski, N. Ziems, H. A. Eicher-Miller, T. J.-J. Li, M. Jiang, and R. A. Metoyer. Limitations of the llm-as-a-judge approach for evaluating llm outputs in expert knowledge tasks. In *Proceedings of the 30th International Conference on Intelligent User Interfaces, IUI '25*, page 952–966, New York, NY, USA, 2025. Association for Computing Machinery.
- [40] F. Tian, D. Ganguly, and C. Macdonald. Is relevance propagated from retriever to generator in rag? In *Advances in Information Retrieval: 47th European Conference on Information Retrieval, ECIR 2025, Lucca, Italy, April 6–10, 2025, Proceedings, Part I*, page 32–48, Berlin, Heidelberg, 2025. Springer-Verlag.
- [41] K. Tian, E. Mitchell, A. Zhou, A. Sharma, R. Rafailov, H. Yao, C. Finn, and C. Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In H. Bouamor, J. Pino, and K. Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5433–5442, Singapore, Dec. 2023. Association for Computational Linguistics.
- [42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [43] C. Wang, R. Ning, B. Pan, T. Wu, Q. Guo, C. Deng, G. Bao, X. Hu, Z. Zhang, Q. Wang, and Y. Zhang. NovelQA: Benchmarking question answering on documents exceeding 200k tokens. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [44] M. Wang, A. Stoll, L. Lange, H. Adel, H. Schütze, and J. Strötgen. Bring your own knowledge: A survey of methods for llm knowledge expansion, 2025.
- [45] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*.
- [46] J. Ye, Y. Wang, Y. Huang, D. Chen, Q. Zhang, N. Moniz, T. Gao, W. Geyer, C. Huang, P.-Y. Chen, N. V. Chawla, and X. Zhang. Justice or prejudice? quantifying biases in LLM-as-a-judge. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [47] X. Zhang, Y. Chen, S. Hu, Z. Xu, J. Chen, M. Hao, X. Han, Z. Thai, S. Wang, Z. Liu, and M. Sun. ∞ Bench: Extending long context evaluation beyond 100K tokens. In L.-W. Ku, A. Martins, and V. Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15262–15277, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics.
- [48] Y. Zhou, Y. Su, Y. Sun, S. Wang, T. Wang, R. He, Y. Zhang, S. Liang, X. Liu, Y. Ma, and Y. Fang. In-depth analysis of graph-based rag in a unified framework, 2025.

A Pseudo Code

Algorithm 1 The pseudo-code for retrieval stage.

Require: $q, \mathcal{G}, T, k, h, l$
Ensure: Supplemental text $\mathcal{C}_s = \{c_1, c_2 \cdots c_n\}, n \leq k$

```

1: entities  $\mathcal{E}_q = \text{SpaCy}(q)$ 
2: if  $\text{Count}(\mathcal{E}_q) == 0$  then
3:   return ★  $\mathcal{C}_s = \text{DenseRetrieval}(q, k)$ 
4: end if
5: selected pairs  $\mathcal{P} = \text{GraphFilter}(\mathcal{E}_q, h)$  [Eq. (1)]
6: if  $\text{Count}(\mathcal{P}) == 0$  then
7:   ★ candidate supplementary chunks  $\hat{\mathcal{C}} = \text{DenseRetrieval}(q, 2k)$ 
8:   return ★  $\mathcal{C}_s = \text{OccurrenceRank}(\hat{\mathcal{C}})$ 
9: end if
10: candidate supplementary chunks  $\hat{\mathcal{C}} = \text{IndexMapping}(\mathcal{P})$  [Eq. (2)]
11: while  $\text{Count}(\hat{\mathcal{C}}) > 25$  do
12:    $h = h - 1$  or  $l = l + 1$ 
13:    $\hat{\mathcal{C}}_{\text{prev}} = \hat{\mathcal{C}}$ 
14:    $\mathcal{P} = \text{GraphFilter}(\mathcal{E}_q, h)$  [Eq. (1)]
15:    $\hat{\mathcal{C}} = \text{IndexMapping}(\mathcal{P})$  [Eq. (2)]
16: end while
17: if  $\text{Count}(\hat{\mathcal{C}}) == 0$  then
18:   return  $\mathcal{C}_s = \text{EntityAwareFilter}(\hat{\mathcal{C}}_{\text{prev}})$ 
19: else
20:   return  $\mathcal{C}_s = \hat{\mathcal{C}}$ 
21: end if
```

B Dataset Details

In this section, we provide detailed descriptions of the datasets used in our experiments. While the main paper introduces the overall dataset choices and their relevance to our task, here we include further information on data statistics.

NovelQA has 89 books along with 2305 multi-choice questions in total, which contain 65 free public-domain books and 24 copyright-protected books purchased from the Internet. It is released with an Apache-2.0 License. **InfiniteChoice** has 58 books along with 229 multi-choice questions in total. **InfiniteQA** has 20 books along with 102 questions in total. The InfiniteBench is released with an MIT License. The links of the two datasets are provided in our code repo, both of the two datasets are publicly accessible.

To provide a deeper insight into our method, we analyze the number of entities involved in each question for all datasets. Specifically, we count the entities mentioned in the question text, excluding those appearing only in the multiple-choice options. The detailed statistics, including the average, minimum, and maximum number of entities per question are reported in Table 3. In addition, Figure 4 illustrates the distribution of questions across different entity count buckets, offering a clearer view of how entity complexity varies across the datasets.

Table 3: Entity count of each question in each dataset

Dataset	NovelQA	InfiniteChoice	InfiniteQA
avg.	4.60	3.24	3.37
min.	0	1	0
max.	24	9	7

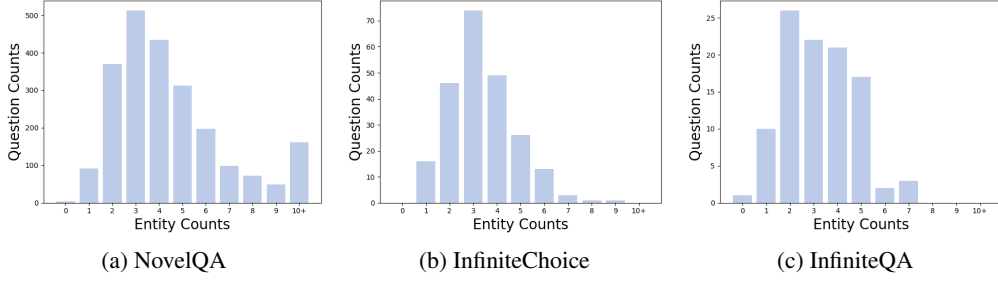


Figure 4: Distribution of questions across different entity count buckets.

C Implementation Details of Baseline Methods

Because of excessive redundant design in the official GraphRAG implementation, we opted for the most widely adopted open-source implementation, nano-GraphRAG, for our experiments. To adapt GraphRAG for local deployment with Huggingface models, we utilized the code from LightRAG that supports Huggingface integration and embedded it into nano-GraphRAG.

For a fair comparison, the hyperparameter settings of all the methods and all baselines are chosen to ensure that the entire pipeline can run on a single NVIDIA A800 GPU with 80GB of memory. For the retrieval level of GraphRAG, we choose the best level, i.e. level 2, reported in the corresponding paper. For the retrieval mode of LightRAG, we choose the Hybrid mode, which is reported as the best mode in the paper. All LLMs are implemented using the HuggingFace `transformers` framework. For our method, the hyperparameter k is determined by the GPU memory and h is automatically changed during the retrieval, therefore we choose a relatively large value 4.

D Proofs for Cost Analysis

The overhead of the retrieval time is relatively trivial, thus we only provide detailed proof of indexing time cost in this section.

E²GraphRAG : Our method builds a tree with n leaf nodes and each non-leaf node has g child nodes. The number of LLM calls is equal to the number of non-leaf nodes. The non-leaf nodes can be listed by level and form a geometric sequence with the first term $\frac{n}{g}$ and the common ratio $\frac{1}{g}$. Therefore, it is a convergent sequence with a converge to $\frac{n}{g} / (1 - \frac{1}{g}) = \frac{n}{g-1}$.

RAPTOR : Similar to our method, RAPTOR builds a tree with n leaf nodes. However, each non-leaf node has at most g child nodes, resulting in the number of non-leaf nodes being larger than E²GraphRAG. Therefore, the **lower bound** is $n/(g-1)$.

LightRAG : LightRAG extracts the entities and relations from each chunk and then assembles them into an entity graph. The extraction phrase takes n times LLM calls, and the assembling phrase does not need the LLM calls. Therefore it requires n times of LLM calls in total.

GraphRAG : GraphRAG extracts the entities and relations from each chunk and formats an entity graph. Then, GraphRAG clusters the nodes and summarizes each community to aggregate the information by LLM. Therefore, the extraction phrase evokes n times LLM calls, and the summarization phrase calls LLM c times, which is $n + c$ in total.

E Limitations

Although we present a streamlined graph-based RAG framework that demonstrates both strong efficiency and effectiveness in this paper, the retrieval design remains relatively intuitive. While we have conducted extensive experiments and explored various alternative retrieval strategies (some

of which are not included in the paper), it is impossible to exhaust all possible retrieval pipeline designs. Therefore, there may still exist more optimal retrieval strategies that could further enhance the performance of our approach.

F Broader Impact

Our work proposes a more efficient and effective graph-based retrieval-augmented generation (RAG) framework, which may benefit downstream applications such as open-domain question answering, knowledge-intensive NLP tasks, and long-document understanding. By significantly reducing the indexing and retrieval cost, our approach could improve the accessibility of large-scale knowledge systems in low-resource or cost-sensitive settings.

However, like other RAG-based systems, our model depends heavily on the quality and neutrality of the underlying documents. If biased or incorrect data are indexed, the system may generate misleading or harmful outputs. Furthermore, automatic entity extraction and graph construction may propagate errors or overlook minority perspectives.

While we do not directly address issues such as fairness or bias mitigation, we encourage responsible use of our framework in conjunction with trustworthy data sources and human oversight. Future work could explore debiasing methods and improved transparency in retrieval paths.